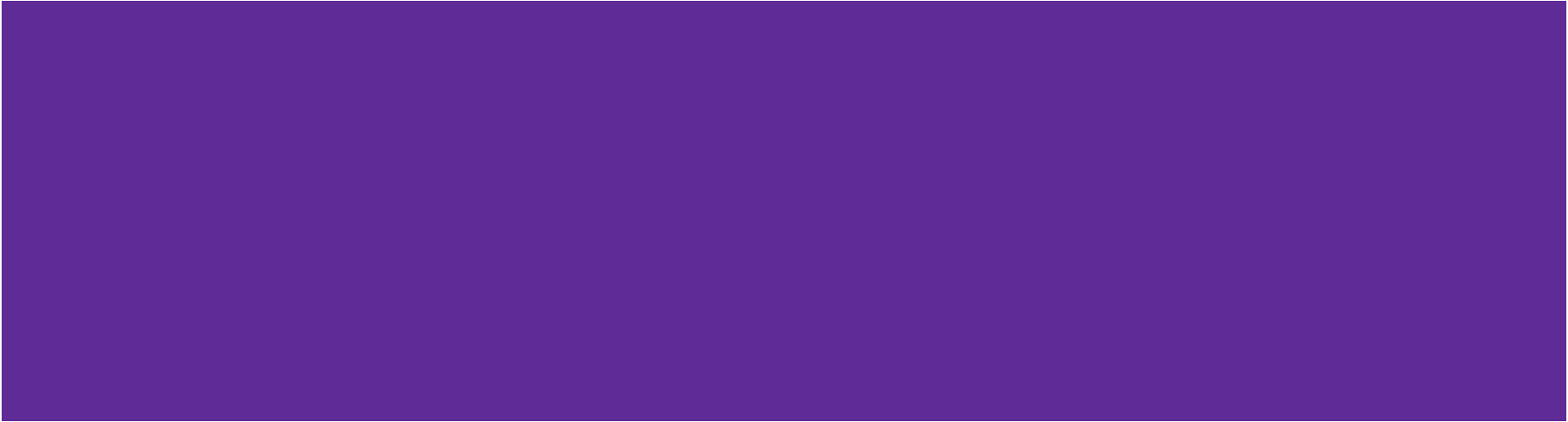


Programación Avanzada

IIC2233 2026-1

Jaime Middleton - Pablo Araneda - Antonio López - Tamara Vidal - Daniela Concha



Experiencia 3

Serialización + Networking II

Experiencia: ¿Qué vamos a hacer?

1. Implementaremos un protocolo de mensajes sobre TCP con el patrón 4 bytes de largo + payload.
2. Construiremos un servidor TCP concurrente con un thread por cliente.
3. Persistiremos el estado del servidor con pickle.
4. Enviaremos objetos serializados con pickle sobre TCP entre el servidor y la GUI.
5. Integraremos un socket no-bloqueante con el event loop de PyQt6.

DCStats

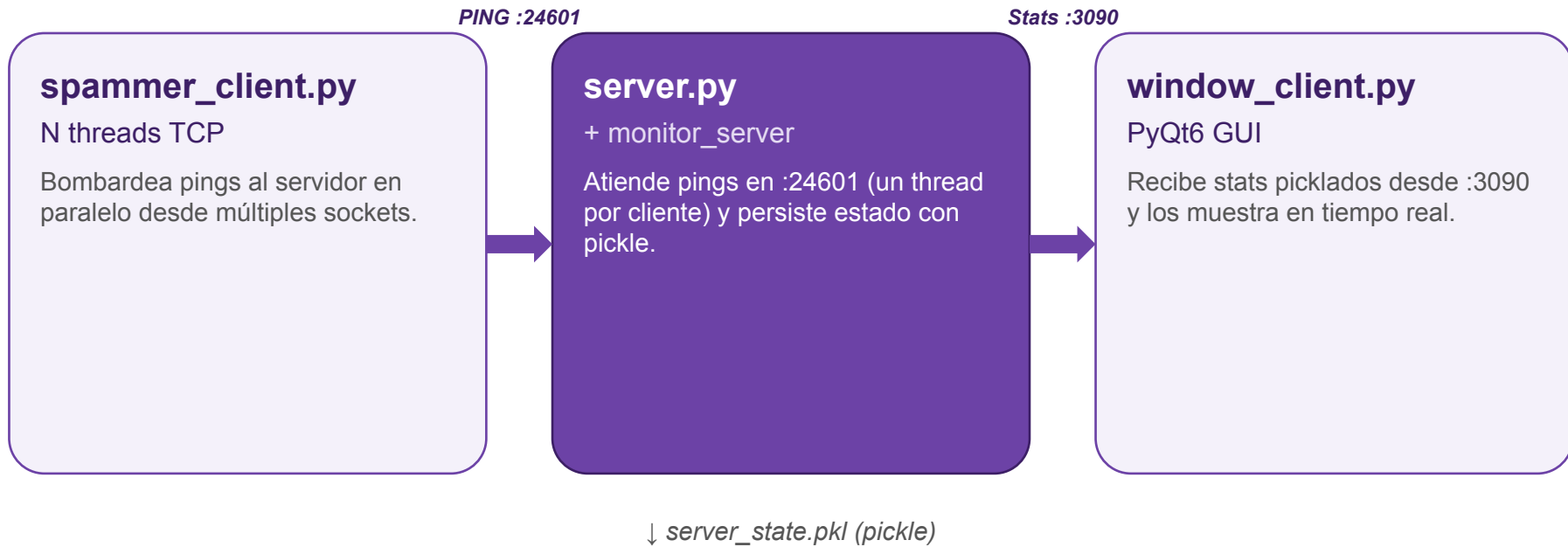


Tu servidor de peticiones está al borde del colapso. Sospechas que algo está saturando el sistema, pero no puedes saberlo sin medirlo en vivo...

Necesitas un monitor de carga en tiempo real.

Arquitectura del sistema

Construiremos un monitor de carga llamado **DCStats**, compuesto por tres procesos que se comunican vía TCP:



server.py

Es el **componente central**. Abre dos sockets TCP en paralelo:

- Puerto **:24601** — recibe pings de los spammers con el patrón **[4B largo][b"PING"]**.
- Atiende **un thread por cliente** (modo daemon).
- Responde **[4B largo][b"OK <total>"]**.
- Persiste su **ServerState** con pickle al cerrar.

Responsabilidades

- Accept loop con threads
- Contadores compartidos (total, window)
- Persistencia con pickle
- Broadcast vía monitor_server

```
$ python server.py
[server] :24601 listening
[server] :3090 broadcasting
```

spammer_client.py

Simula **N clientes concurrentes** bombardeando al servidor:

- Lanza **N threads**; cada uno abre su propio socket TCP a **:24601**.
- Cada thread envía pings en loop con el mismo protocolo de framing.
- Lleva un contador local **sent**.

Para qué sirve

- Generar carga real sobre el servidor
- Probar concurrencia en producción

```
$ python spammer_client.py 20  
[spammer] 20 threads started  
[spammer] sent=1247 ...
```

window_client.py

La **GUI de monitoreo** en PyQt6. Recibe stats y los muestra en vivo:

- Conecta a **:3090** con un socket en **modo no-bloqueante**.
- Un **QTimer** cada **100 ms** llama a **_poll()** para drenar el socket.
- Mantiene un **buffer entre llamadas** para acumular mensajes parciales.
- Despicha cada **Stat** y actualiza los labels de la ventana.

El truco

- `recv()` bloqueante → congela la GUI
- socket no-bloqueante + QTimer 100ms
- Buffer entre tics para mensajes partidos

```
$ python window_client.py  
[gui] connected to :3090  
[gui] total=1247 rps=12.4
```


¿Cómo lo lograremos?

El código de todos los componentes está **incompleto y/o presenta errores**. Hay **5 partes** con un total de **9 sub-tareas** entre TODO y FIXME.

Parte 1 Protocolo de framing

shared.py
3 tareas

Parte 2 Servidor TCP concurrente

server.py
2 tareas

Parte 3 Persistencia con pickle

shared.py
1 tarea

Parte 4 Broadcaster con pickle

monitor_server.py
1 tarea

Parte 5 Cliente GUI no-bloqueante

window_client.py
2 tareas

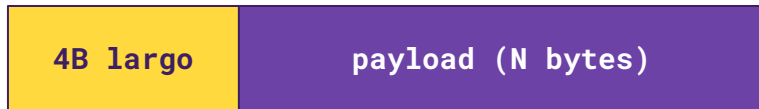
Parte 1.1 — Protocolo de framing

send_message(sock, payload)

TCP es un **stream sin límites de mensaje**: envías “HOLA” y “CHAO”, y el receptor puede leer “HOLACH” + “AO”.

Solución: anteponer un **header de 4 bytes big-endian** con el largo del payload.

Formato del mensaje en la red



¡A programar! 

Parte 1.2 + 1.3 — Recibir con framing

`recv_exactly(sock, n)` · `recv_message(sock)`

1.2 `recv_exactly(sock, n)`

Acumula en un **bytearray** hasta tener exactamente n bytes. Si `recv()` devuelve b'', el peer cerró → lanza **ConnectionError**.

💡 `recv(n)` es el máximo, no exacto. Puedes pedir 4 y recibir 1.

1.3 `recv_message(sock)`

Combina ambas: lee 4 bytes, convierte con `int.from_bytes(...,'big')`, luego lee esa cantidad y retorna el payload.

¡A programar! 

Parte 2.1 — Servidor concurrente

run_accept_loop() — un thread por cliente

Completa el accept loop en **server.py**. Pasos:

- Crear el **socket** y activar **SO_REUSEADDR**.
- Hacer **bind()** y **listen()**.
- En loop infinito: **accept()** y despachar a un thread.
- Usar **threading.Thread(..., daemon=True)**.



Daemon threads se cierran automáticamente al apagar el servidor.

¡A programar!



Parte 2.2 — Race conditions

handle_client() en server.py · worker() en spammer_client.py

Cada conexión corre en su propio thread y toca contadores compartidos.

state.increment() y **_increment('sent')** hacen lectura-modificación-escritura explícita con **time.sleep(0)** para que la carrera sea **visible en segundos**.

Con varios threads concurrentes se **pierden incrementos**.



NO toques increment() ni el cuerpo NO_MODIFICAR.

¡A programar! 

Parte 3 — Persistencia con pickle

ServerState

Necesitamos guardar el estado del servidor en disco entre ejecuciones.

Problema: ServerState contiene un `threading.Lock`, que **no es picklable**.

¡A programar! 

Parte 4 — Broadcaster con pickle

monitor_server._broadcast_loop()

Cada **WINDOW_SECS** (0.1 s), el broadcaster debe:

- Construir **Stats**.
- Llamar a **reset_window**.
- Serializar con **pickle**.
- Enviar a cada cliente con **send_message**.
- Si **OSError**, marcar conn como caído y eliminar después del loop.

🛡️ **No mutes `_clients` mientras iteras → usa una lista `conns_caídos`.**

¡A programar! 🖥️

Parte 5 — Cliente GUI no bloqueante

window_client._poll() con framing y buffer

QTimer llama a `_poll()` cada 100 ms. Dos pasos:

- **Paso 1 (LISTO)** — drenar el socket: leer todo lo que haya con `recv()` no bloqueante y acumular en `self._buf`.
- **Paso 2 (FALTANTE)** — extraer mensajes completos del buffer: mientras haya ≥ 4 bytes, lee el largo y, si hay payload completo, deserializa.

💡 **BlockingIOError** es esperado → significa “no hay más datos”.

💡 **Por qué el buffer?** Un Stats picklado son ~80B; TCP puede partirlo entre tics. El buffer guarda lo parcial.

¡A programar! 

Programación Avanzada

IIC2233 2026-1

Jaime Middleton - Pablo Araneda - Antonio López - Tamara Vidal - Daniela Concha

